

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

Отчет по блоку задач №1 (Тимус)

Выполнил:

Студент группы Р3209

Евграфов А. А.

Преподаватель:

Косяков М. С.

Санкт-Петербург, 2026 г.

Задачи из 1-го блока Тимуса

Задача №1 «2025. Стенка на стенку»

Описание

В соревновании могут участвовать от двух до k команд, каждая из которых будет соперничать с остальными. Всего в соревновании примут участие n бойцов. Перед началом боя они должны разделиться на команды, каждый боец должен войти ровно в одну команду. За время боя два бойца сразятся, если они состоят в разных командах. Требуется максимизировать количество схваток между бойцами.

Ограничение времени: 1 секунда.

Ограничение памяти: 64 Мб.

Решение

В соревновании могут участвовать от двух до k команд, каждая из которых будет соперничать с остальными. Всего в соревновании примут участие n бойцов. Перед началом боя они должны разделиться на команды, каждый боец должен войти ровно в одну команду. За время боя два бойца сразятся, если они состоят в разных командах. Требуется максимизировать количество схваток между бойцами. Сначала заметим, что при увеличении количества долей максимальное количество ребер тоже увеличится, значит мы хотим, чтобы долей было k . Заметим, что тогда ответ $\frac{n(n-1)}{2} - m\frac{x(x-1)}{2} - r\frac{x(x+1)}{2}$, где m - количество долей, в которых будет x вершин, а в r долях будет $x+1$ вершина (причем $mx + (x+1)r = n$ и $m + r = k$).

Доказательство

Сначала докажем, что мы хотим максимизировать количество долей. Действительно, пусть у нас есть граф на $m < k$ долях и в нем проведены всевозможные ребра (с сохранением m -должности). Посмотрим на какую-то долю A , давайте поделим эту долю пополам. Тогда ребра между A и остальными долями останутся, но добавятся ребра между вершинами новых долей.

Пусть размеры долей $x_1, x_2, \dots, x_k$, причем $\sum_{i=1}^k x_i = n$, тогда мы хотим максимизировать количество ребер в графе с долями соответствующего размера. В графе на n вершинах не больше $\frac{n(n-1)}{2}$ рёбер, но в каждой из долей не должно быть ни одного ребра, значит мы хотим

максимизировать величину $\frac{n(n-1)}{2} - \sum_{i=1}^k \frac{x_i(x_i-1)}{2}$ рёбер, а так как n - константа, то

это равносильно минимизации величины $\sum_{i=1}^k \frac{x_i(x_i-1)}{2} = \frac{1}{2} \sum_{i=1}^k x_i^2 - \frac{1}{2} \sum_{i=1}^k x_i = \frac{1}{2} \sum_{i=1}^k x_i^2 - \frac{1}{2}n$,

что в свою очередь равносильно минимизации $\sum_{i=1}^k x_i^2$ при фиксированной сумме $\sum_{i=1}^k x_i$.

Вообще, минимум достигается при равенстве всех x_i , что следует из неравенства между средним квадратическим и средним арифметическим, но покажем это по-другому.

Докажем, что минимум достигается, когда $\forall i, j \in \{1, \dots, k\} |x_i - x_j| \leq 1$.

Пусть для каких-то x_i и x_j $|x_i - x_j| \geq 2$ и $x_i < x_j$, давайте подвигаем эти величины: пусть $x'_i = x_i + 1$, $x'_j = x_j - 1$, тогда сумма всех переменных x не изменилась, $x'_i \leq x'_j$.

$x'^2_i + x'^2_j - x^2_i - x^2_j = 2x_i - 2x_j + 2 = 2(x_i - x_j) + 2 < 0$, значит сумма квадратов уменьшается при приближении двух переменных друг к другу. Аналогично можно показать, что при

удалении переменных друг от друга она будет увеличиваться.

Повторим рассуждения для любых двух переменных, которые отличаются хотя бы на 2, отсюда получается, что все переменные должны отличаться на не более чем 1.

Вернемся к нашей формуле:

$$n = x(k - r) + (x + 1)r = xk + r, \text{ откуда } x = \left\lfloor \frac{n}{k} \right\rfloor$$

Алгоритмическая сложность

- По времени: $O(1)$
- По памяти: $O(1)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int t;
10    cin >> t;
11    while (t--) {
12        long long int n, k;
13        cin >> n >> k;
14        int m = n / k;
15        int r = n % k;
16        long long int res = n * (n - 1) / 2 - (k - r) * m * (m - 1) / 2 - r * m
17        * (m + 1) / 2;
18        cout << res << "\n";
19    }
20
21    return 0;
22 }
```

Задача №2 «1401. Игроки»

Описание

Требуется вывести замещение доски размера $2^n \times 2^n$ уголочками так, чтобы осталась одна незамещенная клетка, если это возможно.

Ограничение времени: 2 секунды.

Ограничение памяти: 64 Мб.

Решение

На самом деле это всегда возможно. Пусть нам нельзя замещать клетку с координатами (x_0, y_0) , будем называть её нераскрашиваемой. Доску будем красить рекурсивно: разделим её на 4 меньших доски, доска с нераскрашиваемой клеткой закрашивается, а для каждой из трех оставшихся досок доопределим нераскрашиваемую клетку так, чтобы в центре начальной доски образовался уголок.

Доказательство

Докажем, что такая раскраска всегда существует для любой нераскрашиваемой клетки. Для $n = 1$ это очевидно, пусть утверждение верно для любой нераскрашиваемой клетки доски $2^{n-1} \times 2^{n-1}$.

Рассмотрим доску $2^n \times 2^n$, разобьем ее на 4 доски размера $2^{n-1} \times 2^{n-1}$, для трех меньших досок, в которых не находится нераскрашиваемая клетка изначальной доски доопределим нераскрашиваемую клетку так, чтобы в центре доски образовался уголок (запретим красить один из углов каждой из трех досок). Доску, где лежит нераскрашиваемая клетка, замостим по предположению индукции, а образованное свободное место в форме уголка замостим последней фигурой. Значит решение существует всегда.

Пусть $N = 2^n$, если $T(N)$ - количество операций для входа размера n , тогда $T(N) = 4T(\frac{N}{2}) + O(1)$, тогда по мастер-теореме $T(N) = N^2$, то есть $T(n) = 4^n$.

Алгоритмическая сложность

- По времени: $O(4^n)$
- По памяти: $O(n)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4 int arr[512][512];
5 int k = 1;
6
7 void solve(int n, int x0, int y0, int x, int y) {
8     if (n == 0) {
9         return;
10    }
11
12    int x1 = x0 + (1 << (n - 1));
13    int y1 = y0 + (1 << (n - 1));
14    int cur = k++;
```

```

15
16 if (x < x1 && y < y1) {
17     arr[x1 - 1][y1] = cur;
18     arr[x1][y1 - 1] = cur;
19     arr[x1][y1] = cur;
20     solve(n - 1, x0, y0, x, y);
21     solve(n - 1, x0, y1, x1 - 1, y1);
22     solve(n - 1, x1, y0, x1, y1 - 1);
23     solve(n - 1, x1, y1, x1, y1);
24 } else if (x < x1 && y >= y1) {
25     arr[x1 - 1][y1 - 1] = cur;
26     arr[x1][y1 - 1] = cur;
27     arr[x1][y1] = cur;
28     solve(n - 1, x0, y0, x1 - 1, y1 - 1);
29     solve(n - 1, x0, y1, x, y);
30     solve(n - 1, x1, y0, x1, y1 - 1);
31     solve(n - 1, x1, y1, x1, y1);
32 } else if (x >= x1 && y < y1) {
33     arr[x1 - 1][y1 - 1] = cur;
34     arr[x1 - 1][y1] = cur;
35     arr[x1][y1] = cur;
36     solve(n - 1, x0, y0, x1 - 1, y1 - 1);
37     solve(n - 1, x0, y1, x1 - 1, y1);
38     solve(n - 1, x1, y0, x, y);
39     solve(n - 1, x1, y1, x1, y1);
40 } else {
41     arr[x1 - 1][y1 - 1] = cur;
42     arr[x1 - 1][y1] = cur;
43     arr[x1][y1 - 1] = cur;
44     solve(n - 1, x0, y0, x1 - 1, y1 - 1);
45     solve(n - 1, x0, y1, x1 - 1, y1);
46     solve(n - 1, x1, y0, x1, y1 - 1);
47     solve(n - 1, x1, y1, x, y);
48 }
49 }
50
51 int main() {
52     ios_base::sync_with_stdio(false);
53     cin.tie(nullptr);
54
55     int n, x, y;
56     cin >> n >> x >> y;
57     solve(n, 0, 0, x - 1, y - 1);
58     arr[x - 1][y - 1] = 0;
59     for (int i = 0; i < (1 << n); i++) {
60         for (int j = 0; j < (1 << n); j++) {
61             cout << arr[i][j] << " ";
62         }
63         cout << "\n";
64     }
65     return 0;
66 }

```